

7

STACK AND QUEUE

7.1 Introduction

Linear data structure in which both insertion and deletion operation are performed at one end called TOP of the stack. Works on LIFO (Last In First Out) Policy.

7.1.1 Application

To post-pone certain decision (To wait/To delay)

- **Stack Permutation :** Order of insertion of key is fixed but we can pop an element any time.

Number of possible stack permutation with n elements = $\frac{{}^{2n}C_n}{n+1}$

- Infix to postfix
- Infix to prefix
- Prefix Evaluation
- Postfix Evaluation
- Recursion
- Tower of Hanoi

7.2 Queue Data Structure

Linear data structure in which insertion is done at one end called rear of the queue and deletion is performed at other end called front end of the queue.

Works on FIFO (First In First Out) policy.

7.2.1 Applications of Queue Data Structure

- Whenever a resource is being shared among many users.
- When data is transferred asynchronously.
- FCFS scheduling
- Algorithms like BFS.

7.3 Implementation

Can be implemented using arrays, linked list.

(a) Using Array :

- Easy to implement and memory efficient.
- Not dynamic

(b) Using Linked List :

- Dynamic
- Require extra memory because of pointer field.

7.4 Standard Operations

7.4.1 Stack

- (a) Push () : Insertion of an element onto stack.
- (b) Pop () : Deletion of an element.
- (c) Is_empty () : Returns true if stack is empty otherwise return false.

7.4.2 Queue

- (a) **Enqueue :** Insertion of an element.
Performed at rear end of the queue.
- (b) **Dequeue :** Deletion of an element.
Performed at front end of the queue.

7.5 Circular Queue

Extended version of simple queue in which last element is connected to the first element.

In simple queue, even though space is available, we are not able to insert a new element and declaring it a overflow. Circular queue solves this problem.

Queue Full

- (i) $\text{Front} == 0 \ \& \ \& \ \text{Rear} == \text{SIZE} - 1$ OR
- (ii) $\text{Front} == (\text{Rear} + 1) \% \text{SIZE}$

7.6 Priority Queue

A queue in which a priority is associated with every element and elements are processed according to their priorities and if two elements have same priority then they are processed as per their arrival in the queue.

